

Continuous-Time State-Space Modelling of the Hot Hand in NBA Field Goals

Candidate Numbers: 63593, 62665

Abstract

Does recent shooting success reflect genuine shifts in performance, or are observers reading patterns into noise? We investigate using NBA field-goal attempts from the 2022–2025 regular seasons, extending a continuous-time state-space model originally developed for free throws to live-play shooting. Latent shooting form follows an Ornstein–Uhlenbeck process, and an expected field-goal adjustment controls for shot difficulty. In-sample, the latent-state model substantially outperforms benchmark logistic regressions both with and without the shot-quality adjustment. Half-lives of 41–55 minutes point to a persistent within-game form effect rather than short-lived make-after-make momentum. On held-out data, though, the model does not improve next-shot prediction: shot outcomes alone carry too weak a signal to be useful in real time.

1 Introduction

The hot-hand effect is one of the most familiar and debated ideas in basketball. It refers to the belief that a player who has made several shots in a row is more likely to make the next one, often described as being “hot” or “in the zone”. This idea matters because it can shape in-game decisions, including whether teams continue directing possessions toward a player on a run and whether defenders respond differently to recent shooting success. More broadly, the hot hand raises a statistical question about human performance: do players move through genuine periods of unusually high and low performance, or do observers over-interpret random variation? (Gilovich et al., 1985)

This paper investigates the hot hand in NBA field-goal shooting by modelling whether players exhibit time-varying shooting form during games. We build on the continuous-time state-space framework of Mews and Ötting (2023), who apply an Ornstein–Uhlenbeck latent-state model to NBA free throws. We extend this framework to field goals, where shot success is affected by shot selection, defensive pressure, game context, and player identity. Since field goals vary substantially in difficulty, we also incorporate an expected field-goal measure, following the broader motivation of Kondur and Shen (2025), who show that accounting for shot difficulty is central when studying the hot hand in live-play shooting.

The paper has three aims. It first tests whether a continuous latent-state model improves the explanation of NBA field-goal outcomes relative to a benchmark model without time-varying form. It then examines whether this evidence

remains after adjusting for shot difficulty through an expected field-goal model. Finally, it evaluates whether the fitted models improve out-of-sample prediction on unseen data. Together, these aims assess whether the hot-hand framework can be meaningfully extended from the controlled free-throw setting to the more complex environment of live-play field goals.

The remainder of the paper is organised as follows. Section 1.1 reviews the literature on the hot-hand effect, from the early debate to more recent latent-state and shot-difficulty approaches. Section 2 describes the dataset and methodological framework. Section 3 presents the empirical results. Section 4 discusses the findings in relation to the wider literature, considers the main limitations, and draws the paper’s conclusions.

1.1 Literature Review

The hot-hand effect refers to the claim that a player is more likely to make a shot after a sequence of made shots. The modern debate is usually traced to Gilovich et al. (1985), who analysed field-goal data from the Philadelphia 76ers, free-throw data from the Boston Celtics, and evidence from a controlled shooting experiment. They found no positive dependence between successive shots and argued that belief in the hot hand reflected a tendency to perceive streaks in random sequences. One limitation of this approach is that it treats the hot hand mainly as direct shot-to-shot dependence. A player may instead experience a broader period of elevated or diminished performance, which need not appear as a simple make-after-make effect.

Subsequent work questioned both the conclusion and the methods used to reach it. Arkes (2010) found evidence of a small free-throw hot hand in NBA data using a multivariate model with player fixed effects, while Yaari and Eisenmann (2011) reported non-random patterns in free-throw sequences and interpreted them as possible evidence of good and bad performance periods rather than immediate shot-to-shot momentum. These studies improved on simple streak comparisons by allowing for player-level differences, but they still relied mainly on observed outcome histories. The measurement problem became more serious after Miller and Sanjurjo (2018), who showed that common streak-based tests are affected by selection bias. Even when shots are

independent, conditioning on previous streaks can distort the observed success rate on the next shot. The issue is therefore not only whether streaks appear in the data, but whether the procedure used to identify them is valid.

More recent studies have moved beyond simple streak comparisons. Chang (2019) used Bayesian multistep Markov chain methods for free throws and found statistical dependence in a minority of player-seasons. Lantis and Nesson (2021) examined free throws and field goals across twelve NBA seasons, finding a small hot-hand effect for free throws, especially after longer made-shot streaks, but not the same pattern for field goals. This distinction matters because free throws are relatively standardised, whereas field goals are affected by shot selection, defensive pressure, player movement, and game context. Methods that are persuasive for free throws may therefore be less convincing for field goals unless they account for the changing difficulty of each attempt.

A different approach treats the hot hand as variation in an unobserved performance state rather than as dependence between adjacent outcomes. Wetzels et al. (2016) compare a constant-performance model with a two-state hidden Markov model, allowing players to switch between latent performance regimes. This recognises that shooting form is not directly observed, although the two-state structure may be too coarse if performance changes gradually. Mews and Ötting (2023) develop this idea further by modelling latent shooting form as an Ornstein–Uhlenbeck process in continuous time. This is useful in basketball because attempts are irregularly spaced within games. However, their application is to free throws, so it avoids the main complication of live-play shooting: field goals vary substantially in baseline difficulty. Calvo et al. (2025) also use a latent-state framework at team level, although a team-level state has a different interpretation from individual shooting form.

Shot difficulty is therefore central when extending latent-state methods to field goals. If variation in shot quality is ignored, a model may attribute easier or harder attempts to changes in latent shooting form. Kondur and Shen (2025) show that controlling for shot distance, player characteristics, and defender proximity changes the interpretation of hot-hand evidence, with traditional effects becoming more limited once shot difficulty is taken into account. This motivates the approach taken in this paper. We adopt the continuous-time latent-state framework of Mews and Ötting (2023), apply it to NBA field-goal data, and extend the observation equation with an expected field-goal measure. This allows us to examine whether time-varying shooting form remains after adjusting for estimated shot quality.

2 Data and Methodological Framework

This section describes the data and modelling framework used to investigate time-varying shooting form in NBA field goals.

We first explain how the play-by-play data were filtered and converted into player-game shooting sequences. We then introduce the baseline continuous-time state-space model, before extending it with an expected field-goal measure to account for shot difficulty. Finally, we describe the likelihood approximation, out-of-sample testing procedure, and the metrics used to evaluate model performance.

2.1 Data Description

The data used in this study consist of NBA shot-level records from the 2022, 2023, 2024, and 2025 regular seasons. The data were obtained from the public `nba_data` GitHub repository, which compiles NBA play-by-play and shot-detail data from sources including `stats.nba.com`, `data.nba.com`, and `pbpstats.com` (Shufinskiy, 2025). The original play-by-play data were filtered to retain only shot-related events. Non-shot actions, including fouls, time-outs, substitutions, rebounds, steals, and turnovers, were removed because the analysis focuses on shooting performance rather than all in-game events.

Each row in the cleaned dataset represents a single shot attempt. The data contains game, player, team, timing, context, location, and outcome information. This includes identifiers for the player and game, elapsed time within the game, home status, score difference, shot type, shot distance, court co-ordinates, and whether the shot was made or missed. This structure gives a shot-level panel suitable for modelling how shooting outcomes vary across players, games, and time.

For the state-space models, the data were organised into player-game shooting sequences. Each sequence consists of all field-goal attempts taken by a given player in a given game, ordered by elapsed game time. This produces an irregularly spaced sequence of made and missed shots, which is required because the continuous-time model treats shooting performance as evolving within each player-game sequence.

2.2 Descriptive Analysis and Sample Construction

Before fitting the state-space models, we summarise how the modelling sample is constructed. Table 1 reports the number of shots, player-game sequences, players, and make rates retained at each filtering stage under the primary four-shot sequence threshold.

Table 1: Construction of the primary modelling sample using the four-shot sequence threshold.

Stage	Shots	Seq.	Players	Make
Training pool	570,067	64,943	768	0.470
After player filter	506,759	50,321	342	0.473
After 4-shot filter	493,888	44,417	342	0.474
Held-out pool	87,313	10,126	549	0.475
Held-out player filter	67,051	6,773	309	0.476
Held-out 4-shot filter	65,356	5,988	308	0.477

Table 1 reports the construction of the primary modelling sample. The player-level filter reduces the training sample

from 768 to 342 players while retaining most shot attempts, and the four-shot sequence filter leaves 493,888 training shots across 44,417 player-game sequences. The make rate remains stable across filtering steps, suggesting that the sample restrictions do not materially change the overall outcome distribution.

The sequence-length distribution is important because the OU model updates the latent shooting state within each player-game sequence. Figure 1 shows that player-game sequences in the player-filtered training sample typically contain moderate shooting volume, with a median of 9 shots and an interquartile range from 6 to 14 shots.

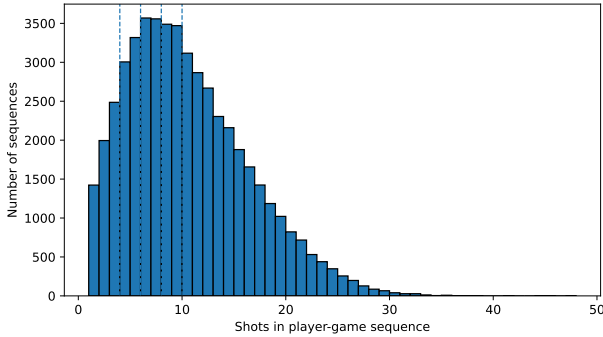


Figure 1: Distribution of player-game sequence lengths in the training sample after applying the player attempt filter. Dashed vertical lines indicate the 4-, 6-, 8-, and 10-shot sequence thresholds used in the main and robustness specifications.

This motivates the use of the four-shot threshold as the primary specification, since it retains a large share of the available player-game sequences. The six-, eight-, and ten-shot thresholds are treated as robustness checks on sequences with richer in-game shooting histories.

2.3 Baseline Continuous-Time State-Space Model

Our baseline model follows the continuous-time state-space framework of Mews and Ötting (2023), but applies it to NBA field-goal attempts rather than free throws. For player p in game n , we observe shot outcomes at irregular times $t_1, \dots, t_T$. The binary outcome is

$$Y_{t_k}^{p,n} \sim \text{Bernoulli}(\pi_{t_k}^{p,n}),$$

where $Y_{t_k}^{p,n} = 1$ if the shot is made and 0 otherwise.

The baseline observation equation is

$$\text{logit}(\pi_{t_k}^{p,n}) = S_{t_k}^{p,n} + \beta_{0,p} + \beta^\top \mathbf{x}_{t_k}^{p,n},$$

where $\mathbf{x}_{t_k}^{p,n} = (\text{home}, \text{scorediff}, \text{last30})$. The player-specific intercept $\beta_{0,p}$ captures differences in average shooting ability and the covariates control for basic game context. The latent state $S_{t_k}^{p,n}$ represents the player's unobserved shooting form at the time of the shot.

Following Mews and Ötting (2023), the latent state is modelled as an Ornstein–Uhlenbeck process:

$$dS_t = \theta(\mu - S_t) dt + \sigma dB_t, \quad \mu = 0,$$

where B_t is a standard Brownian motion. The parameter $\theta > 0$ controls the speed of mean reversion, so smaller values imply that deviations from average form persist for longer. The parameter $\sigma > 0$ controls the volatility of the latent state. Under this specification, shooting form is allowed to change over game time rather than remain fixed across all attempts.

As in the original framework, the latent state is restricted to $[-2, 2]$ for estimation. The main parameters of interest are θ and σ . The model is fitted using the regular-season data from the 2023 season, the 2024 season, and the first 60% of the 2025 season.

2.4 Shot-difficulty Adjustment and xFG-Extended Model

Field goals are much less standardised than free throws. A layup, dunk, contested mid-range shot, and three-point attempt have different baseline probabilities of success. If this variation is ignored, the model may attribute shot difficulty to latent shooting form. This is a central issue when moving from free throws to field goals, since live-play attempts are shaped by shot location, defensive pressure, player movement, and game context (Kondur & Shen, 2025; Mews & Ötting, 2023).

To address this, we estimate an expected field-goal model, denoted xFG , using regular-season field-goal attempts from the 2022 season. The response is whether the shot was made, and the predictors include shot distance, court location, shot type, action description, home status, and whether the attempt occurred in the final 30 seconds of a period. Categorical shot descriptions are grouped into broader shot-type and action-type categories before fitting the model.

The xFG model is a logistic regression,

$$\text{logit}(q_i) = \alpha + \gamma^\top \mathbf{z}_i,$$

where q_i is the make probability for shot i , and $\mathbf{z}_i$ contains the observed shot-quality variables. The fitted value $\hat{q}_i$ is the expected field-goal probability. Using a 0.5 classification threshold, the fitted model achieves an in-sample accuracy of 0.621 on the 2022 field-goal data. This suggests that the xFG model captures meaningful shot-quality variation, but it does not fully validate the predicted probabilities because accuracy does not assess calibration and the evaluation is in-sample. The accuracy is similar in magnitude to that reported by Kondur and Shen (2025), although the comparison is not direct because the models use different data and features.

As an additional out-of-sample descriptive check, Figure 2 evaluates the fitted xFG model on 2023 field-goal

attempts. The figure compares mean predicted xFG with the observed make rate within deciles of predicted shot difficulty.

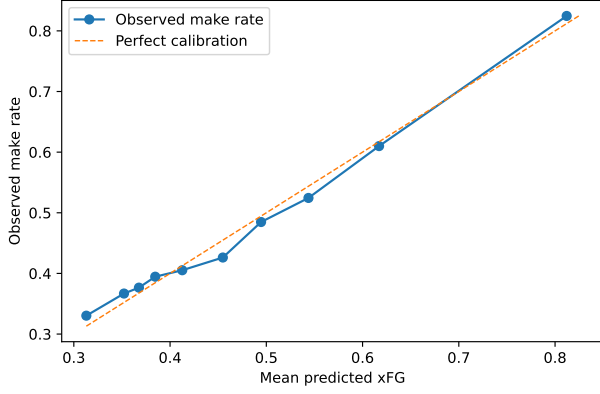


Figure 2: Observed make rate by expected field-goal decile on 2023 field-goal attempts. The expected field-goal model is trained on 2022 data and evaluated on 2023 data. Each point reports the mean predicted xFG and observed make rate within a decile of predicted shot difficulty. The dashed line represents perfect calibration.

Observed make rates increase steadily across xFG deciles, from 0.330 in the lowest decile to 0.825 in the highest decile. The points also lie close to the perfect-calibration line, although the model is not perfectly calibrated in every decile. This supports using xFG as an approximate shot-quality adjustment, while reinforcing that it does not fully capture all determinants of field-goal difficulty.

Since the state-space observation equation is on the logit scale, we use the logit-transformed expected field-goal probability,

$$xFG_i^{\text{logit}} = \text{logit}(\hat{q}_i).$$

The extended observation equation is

$$\text{logit}(\pi_{t_k}^{p,n}) = S_{t_k}^{p,n} + \beta_{0,p} + \beta_1 xFG_{t_k}^{\text{logit},p,n} + \beta_2 \text{scorediff}_{t_k}^{p,n}.$$

Here, $xFG_{t_k}^{\text{logit},p,n}$ controls for shot difficulty, while $S_{t_k}^{p,n}$ captures remaining time-varying shooting form. A difficult make therefore provides stronger evidence of a high latent state than an easy make, while a difficult miss is less damaging to the inferred state than an easy miss.

All other aspects of the state-space model are unchanged. The latent state still follows the Ornstein–Uhlenbeck process with $\mu = 0$, the state space is discretised over $[-2, 2]$, and the main parameters of interest remain θ and σ . This extension tests whether time-varying shooting form remains useful after accounting for estimated shot quality.

2.5 Likelihood Approximation and Model Estimation

Following the likelihood approximation used by Mews and Ötting (2023) and the maximum approximate likelihood procedure developed in Mews et al. (2024), we evaluate the likelihood by discretising the latent state space. For a sequence with T shot attempts, the likelihood involves integrating over the unobserved states $S_{t_1}, \dots, S_{t_T}$, since the shooting state is continuous-valued and not directly observed. As in Mews and Ötting (2023), this T -dimensional integral is approximated by placing a finite grid over the state space, here taken to be $[-2, 2]$. This gives a finite-state approximation of the continuous latent process, so the likelihood can be computed using the forward algorithm, following the maximum approximate likelihood framework of Mews et al. (2024).

The transition probabilities between latent states are determined by the Ornstein–Uhlenbeck process. If $\Delta_k = t_k - t_{k-1}$ is the time between consecutive shots, then

$$S_{t_k} | S_{t_{k-1}} = s \sim N\left(e^{-\theta\Delta_k}s, \frac{\sigma^2}{2\theta}(1 - e^{-2\theta\Delta_k})\right).$$

This transition distribution allows the model to account for irregular timing. Shots taken close together have more strongly related latent states, while longer gaps allow more reversion toward average form.

The approximate likelihood is evaluated separately for each player-game sequence and then combined across sequences, assuming independence conditional on the model parameters. Parameters are estimated by minimising the negative log-likelihood. This procedure is applied to both the baseline and xFG-adjusted specifications, allowing us to test whether the latent OU state improves model fit under each formulation.

2.6 Out-of-Sample Testing

Out-of-sample performance is evaluated using the final 40% of the 2025 season, which is held out from model estimation. As mentioned previously, the training sample consists of the 2023 season, the 2024 season, and the first 60% of the 2025 season. The fitted OU model and the corresponding no-state benchmark are then evaluated on the held-out 2025 data without re-estimating their parameters.

The test data is organised into the same player-game shooting sequences used in estimation. Only players observed in the training sample are retained, and the same minimum sequence-length threshold is applied.

Predictions are made sequentially for each held-out sequence. For the first shot in a sequence, the OU model predicts from the stationary prior centred at zero. For shot $k > 1$, the model uses only shots $1, \dots, k-1$ from that sequence to form the filtered distribution after the previous

observation. This distribution is then propagated to the time of shot k using the fitted OU transition dynamics, and the model integrates over the resulting predictive state distribution to obtain the probability of making shot k . After the outcome of shot k is observed, the filtered state distribution is updated and carried forward to the next shot time.

The benchmark model is a logistic regression with the same observed covariates as the corresponding OU specification but without the latent state. This allows the out-of-sample comparison to isolate whether the sequentially updated latent state improves next-shot prediction beyond the observed predictors.

Predictive performance is assessed using test negative log-likelihood and Brier score. Lower values indicate better forecasts. We report results both for all held-out shots and for shots after the first attempt in each sequence. The latter is the cleaner test of the latent-state mechanism, because the OU model can only use previous within-sequence shot outcomes after at least one shot has been observed.

2.7 Model evaluation and interpretation

Model performance is assessed by comparing each OU state-space model with its corresponding benchmark model without a latent state. The main fit measure is the negative log-likelihood (NLL). Since the OU model introduces additional latent-state structure, we also report the Akaike Information Criterion and Bayesian Information Criterion (Akaike, 1974; Schwarz, 1978):

$$\begin{aligned} \text{AIC} &= 2k - 2 \log(\hat{L}), \\ \text{BIC} &= k \log(n) - 2 \log(\hat{L}), \end{aligned}$$

where k is the number of estimated parameters, n is the number of observations, and $\hat{L}$ is the maximised likelihood. Lower values indicate better fit after penalising model complexity.

Evidence for time-varying shooting form is strongest when the OU model improves on the benchmark under the same covariate specification. The key comparison is therefore between the xFG -adjusted benchmark model and the xFG -adjusted OU model. If the OU model has lower NLL, AIC, and BIC after controlling for shot difficulty, this suggests that field-goal success contains temporal structure beyond observable shot quality.

The OU parameters are used to interpret the latent shooting state. The drift parameter θ measures the speed of mean reversion: smaller values imply more persistent shooting form, while larger values imply faster reversion. The diffusion parameter σ measures the volatility of the latent state, with larger values indicating greater short-run variation.

To make persistence easier to interpret, we report the

half-life of the latent state:

$$\text{Half-life} = \frac{\log(2)}{\theta}.$$

This measures how long it takes for half of a deviation from average form to disappear. We also report the stationary standard deviation of the OU process,

$$\sqrt{\frac{\sigma^2}{2\theta}},$$

which measures the long-run variability of the latent state around its mean. Together, NLL, AIC, BIC, half-life, and stationary standard deviation allow us to assess both model fit and the basketball interpretation of the latent-state process.

3 Empirical Results

This section reports the main empirical findings. We first compare the baseline OU model against its no-state logistic benchmark, then repeat the comparison after adding the xFG adjustment. Improvements are defined as benchmark minus OU, so positive values of ΔNLL , ΔAIC , and ΔBIC favour the OU model. The final subsection reports held-out next-shot prediction on the final 40% of the 2025 season.

3.1 Baseline Model without Shot-Difficulty Adjustment

Table 2: Baseline model without shot-difficulty adjustment. Positive improvements favour the OU model.

Min.	Shots	Seq.	$\hat{\theta}$	$\hat{\sigma}$	Stat. SD	Half-life(min)	ΔNLL	ΔAIC	ΔBIC
4	493888	44417	0.015	0.108	0.629	47.35	3594	7184	7162
6	465278	38094	0.014	0.108	0.649	50.53	4075	8145	8123
8	418958	30967	0.013	0.109	0.677	53.36	4666	9328	9306
10	359807	24007	0.013	0.112	0.707	55.39	5185	10367	10345

Table 2 shows that the OU model outperforms the benchmark at every sequence-length threshold. Under the primary four-shot specification, it lowers NLL by 3594, with AIC and BIC improvements of 7184 and 7162, respectively. Since both criteria penalise model complexity, the improvement suggests that the latent-state component captures variation in shooting performance that is not explained by the benchmark logistic model.

The estimated drift parameter is small throughout, with $\hat{\theta}$ ranging from 0.013 to 0.015. The corresponding half-lives of roughly 47 to 55 minutes imply slow reversion toward average form, while the stationary standard deviation rises from 0.629 to 0.707 as the sequence threshold increases. The baseline model therefore provides initial evidence of persistent temporal structure, although some of this may reflect unmodelled shot difficulty.

3.2 Model with Shot-Difficulty Adjustment

Table 3: xFG -adjusted model results. Positive improvements favour the OU model over the benchmark model.

Min	Shots	Seq.	$\hat{\beta}_{xFG}$	$\hat{\theta}$	$\hat{\sigma}$	Stat. SD	Half-life(min)	Δ NLL	Δ AIC	Δ BIC
4	493888	44417	0.764	0.018	0.116	0.633	41.27	3385	6765	6743
6	465278	38094	0.771	0.016	0.115	0.646	43.37	3806	7606	7585
8	418958	30967	0.728	0.014	0.111	0.673	51.09	4337	8669	8648
10	359807	24007	0.723	0.014	0.112	0.700	49.35	4755	9508	9486

Table 3 shows that the OU model remains preferred after controlling for shot difficulty. In the primary specification, Δ NLL falls from 3594 to 3385 after adding xFG , and Δ AIC falls from 7184 to 6765. This attenuation suggests that shot quality explains part of the baseline temporal structure, but the remaining improvement is still substantial.

The estimated xFG coefficient is positive across all thresholds, ranging from 0.723 to 0.771. The OU parameters also remain close to the baseline estimates: $\hat{\theta}$ ranges from 0.014 to 0.018, with half-lives of approximately 41 to 51 minutes and stationary standard deviations between 0.633 and 0.700. Thus, the xFG -adjusted model provides the cleaner in-sample evidence that field-goal success contains a persistent latent component beyond estimated shot quality.

The fit improvements increase at higher sequence thresholds, although these thresholds also select a different set of player-games. We therefore treat the four-shot threshold as the primary specification and the higher thresholds as robustness checks among richer shooting sequences.

3.3 Out-of-Sample Testing

Table 4: Out-of-sample prediction on held-out 2025 data, minimum 4 shots per sequence. Negative values indicate worse performance for the OU model.

Specification	Evaluation set	Δ NLL	Δ NLL/shot	Δ Brier
No xFG	All shots	−806	−0.0123	−0.0047
No xFG	After first	−752	−0.0127	−0.0048
xFG -adjusted	All shots	−686	−0.0105	−0.0040
xFG -adjusted	After first	−636	−0.0107	−0.0041

All-shots test set: 65,356 shots across 5,988 sequences from 308 players.
After-first test set: 59,368 shots, excluding the first shot from each sequence.

Table 4 shows that the OU model performs worse than the benchmark on held-out 2025 data. This holds with and without the xFG adjustment, and also after excluding the first shot of each sequence. The latter check is important because the OU model has then observed at least one previous shot from the same player-game sequence, yet still fails to improve prediction.

The xFG -adjusted benchmark performs much better than the baseline benchmark, reducing all-shot test NLL from 47,741 to 44,963. Shot difficulty is therefore strongly predictive out of sample, but the sequentially updated OU state

adds no further predictive value. The deterioration is small per shot, around 0.010–0.012 NLL, but consistent across NLL and Brier score.

This separates retrospective fit from real-time prediction. In-sample, the OU model explains temporal structure in completed sequences. Out of sample, each player-game begins from a prior centred at zero and the model must infer the state from only the shots observed so far. The same pattern also holds at higher sequence thresholds, suggesting that time-varying form may be detectable historically but difficult to exploit for next-shot prediction using shot outcomes alone.

4 Discussion of Results

The continuous-time latent-state framework of Mews and Ötting (2023) provides the closest methodological comparison, although their setting is NBA free throws rather than field goals. They estimate $\theta = 0.042$, $\sigma = 0.101$, and a stationary standard deviation of 0.348. Our primary baseline field-goal specification gives $\hat{\theta} = 0.015$, $\hat{\sigma} = 0.108$, and a stationary standard deviation of 0.629. The diffusion estimates are similar, but the smaller drift parameter implies a much longer half-life: approximately 47 minutes in our baseline model compared with about 16.5 minutes in their free-throw model. The fitted field-goal latent state is therefore more persistent and more variable than the corresponding free-throw state.

This difference is plausible because field goals are fundamentally less controlled than free throws. Free throws are standardised, isolated attempts, whereas field goals arise from live play and are shaped by shot creation, defensive pressure, fatigue, movement, and offensive role. The large information-criteria improvements in our baseline model support the presence of temporal structure, although they should not be read as directly comparable effect sizes because the sample size and shooting environment differ substantially.

Shot difficulty is the central issue in extending the model to field goals. The xFG adjustment reduces the primary Δ AIC from 7184 to 6765, suggesting that some of the baseline temporal structure was due to variation in shot quality. The OU model nevertheless remains strongly preferred after this adjustment, and this is the cleanest in-sample result of the study: field-goal outcomes contain persistent temporal structure that is not fully explained by the observed shot-difficulty variables.

Our findings here also speak to Kondur and Shen (2025), who argue that traditional hot-hand effects become limited once shot difficulty is controlled for. We partly agree, since adjusting for xFG weakens the latent-state improvement without eliminating it. One reason for the residual effect is methodological: streak-based tests focus on immediate outcome dependence after recent makes, while the continuous-time framework estimates a smoother latent process evolving

over game time. A state with a half-life of roughly 41 to 55 minutes need not produce a sharp make-after-make effect.

The estimated half-lives suggest that the model is not primarily detecting a short-lived “hot streak”. It is closer to a broader within-game rhythm or “hot game” component, which fits the field-goal setting where performance may reflect sustained offensive involvement, matchups, and physical exertion. The latent state is best understood as a statistical summary of time-varying success probabilities rather than direct proof of a psychological hot-hand mechanism, since binary shot outcomes are noisy signals of form and persistent unobserved shot difficulty could inflate the estimated persistence.

These findings suggest that the traditional hot-hand concept may be too narrow for live-play basketball. The classic definition focuses on repeated individual shot-making, but basketball performance also includes passing, spacing, screening, defence, and decision-making. Opponents can respond strategically by shifting defensive attention, while teammates may alter shot allocation. Observed field-goal outcomes therefore combine form, role, shot selection, and opponent response rather than repeated performance on a controlled task. This helps explain why the hot-hand framework may be more natural in standardised settings such as free throws, while live-play field goals are better interpreted through a broader “hot game” lens: persistent within-game performance variation may exist, but it is not well captured by immediate make-after-make momentum or by shot outcomes alone.

The out-of-sample results add an important qualification. Although the OU model improves in-sample fit, it performs worse than the benchmark on held-out 2025 data, both with and without xFG . This remains true even when the first shot of each sequence is excluded, so the result is not driven only by the model starting each sequence from an uninformative prior. Retrospectively, fitted-state inference can use an observed sequence to identify periods of elevated or diminished latent form. In real time, each new player-game sequence begins from a prior centred at zero, and early shot outcomes provide only weak information about the latent state. The model adds estimation noise before it has enough within-game evidence to improve prediction.

Overall, our results sit between the older sceptical view of the hot hand and stronger claims that recent success is directly actionable. Persistent time-varying form appears visible in historical field-goal data and survives shot-quality adjustment, but the real-time signal from shot outcomes alone is too weak to improve next-shot prediction. Practically, this suggests that coaches and analysts should be cautious about treating recent makes as reliable evidence of current form unless richer real-time information is available.

4.1 Limitations and Directions for Future Research

The main limitation is the quality and granularity of the available data. Although the dataset contains detailed shot-level information, it lacks tracking variables such as defender distance, shot-clock time, player speed, defensive pressure, shot angle, and the spatial positioning of all players. These variables are likely to be important determinants of field-goal difficulty. Our xFG model therefore adjusts for shot quality only approximately, and proprietary tracking data would likely give a cleaner separation between shot difficulty and latent player form.

A second limitation is that the hot hand is defined only through shooting success. Basketball performance is multi-dimensional, and players can contribute through playmaking, rebounding, defence, spacing, screening, and ball security. These dimensions are not included in the state-space model because the analysis focuses on shot outcomes. Future work could broaden the definition of hot hand beyond scoring, or fit role-specific models if reliable positional information were available.

A complementary direction is to apply the same continuous-time framework to sports with more controlled and isolated repeated actions, such as baseball pitching or batting, tennis serving, or golf. These settings reduce the influence of teammates, defensive schemes, and shot selection, which would allow the latent state to be estimated against a cleaner outcome process.

Finally, the results should not be interpreted causally. Field-goal attempts are not randomly assigned, and shot difficulty may itself respond to latent form. A player in elevated form may take harder shots, while opposing defences may contest them more aggressively. Adjusting for shot difficulty may therefore remove part of the signal of interest, while incomplete adjustment may leave residual difficulty inside the latent state. The results are best read as evidence of persistent temporal structure in field-goal success probabilities, not as proof of a pure psychological hot-hand effect.

4.2 Conclusion

This study set out to test whether the hot-hand framework can be extended from NBA free throws to field-goal shooting. The results show that a continuous-time latent-state model improves in-sample fit both before and after adjusting for shot difficulty, suggesting that field-goal outcomes contain persistent temporal structure beyond observable shot quality. However, the same latent state does not improve held-out next-shot prediction, indicating that this structure is easier to detect retrospectively than to use in real time.

Overall, the findings support a more cautious interpretation of the hot hand in live-play basketball. The evidence points less toward short-lived make-after-make momentum

and more toward a slower within-game form component, or “hot game”, shaped by shot quality, player role, defensive response, and game context. The hot hand may therefore be too narrow a concept for field-goal shooting unless richer tracking data can separate genuine changes in form from the strategic and contextual features of live play.

References

- Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716–723. <https://doi.org/10.1109/TAC.1974.1100705>
- Arkes, J. (2010). Revisiting the hot hand theory with free throw data in a multivariate framework. *Journal of Quantitative Analysis in Sports*, 6(1), 1–12. <https://doi.org/10.2202/1559-0410.1198>
- Calvo, G., Armero, C., & Spezia, L. (2025). Can the hot hand phenomenon be modelled? A Bayesian hidden Markov approach. *Computational Statistics*, 40, 2195–2222. <https://doi.org/10.1007/s00180-024-01560-8>
- Chang, J. C. (2019). Predictive Bayesian selection of multistep Markov chains, applied to the detection of the hot hand and other statistical dependencies in free throws. *Royal Society Open Science*, 6(3), 182174. <https://doi.org/10.1098/rsos.182174>
- Gilovich, T., Vallone, R., & Tversky, A. (1985). The hot hand in basketball: On the misperception of random sequences. *Cognitive Psychology*, 17(3), 295–314. [https://doi.org/10.1016/0010-0285\(85\)90010-6](https://doi.org/10.1016/0010-0285(85)90010-6)
- Kondur, A., & Shen, W. (2025). An evaluation of the hot-hand hypothesis in the NBA when accounting for shot difficulty [Advance online publication]. *International Journal of Sports Science & Coaching*. <https://doi.org/10.1177/17479541251374788>
- Lantis, R. M., & Nesson, E. T. (2021). Hot shots: An analysis of the “hot hand” in NBA field goal and free throw shooting. *Journal of Sports Economics*, 22(6), 639–677. <https://doi.org/10.1177/15270025211006889>
- Mews, S., Langrock, R., Ötting, M., Yaqine, H., & Reinecke, J. (2024). Maximum approximate likelihood estimation of general continuous-time state-space models. *Statistical Modelling*, 24(1), 9–28. <https://doi.org/10.1177/1471082X211065785>
- Mews, S., & Ötting, M. (2023). Continuous-time state-space modelling of the hot hand in basketball. *AStA Advances in Statistical Analysis*, 107(1–2), 313–326. <https://doi.org/10.1007/s10182-021-00410-y>
- Miller, J. B., & Sanjurjo, A. (2018). Surprised by the hot hand fallacy? A truth in the law of small numbers. *Econometrica*, 86(6), 2019–2047. <https://doi.org/10.3982/ECTA14943>
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461–464. <https://doi.org/10.1214/aos/1176344136>
- Shufinskiy, A. (2025). *NBA play-by-play data and shot details from 1996/97 to 2024/25*. Retrieved April 30, 2026, from https://github.com/shufinskiy/nba_data
- Wetzels, R., Tutschkow, D., Dolan, C. V., van der Sluis, S., Dutilh, G., & Wagenmakers, E.-J. (2016). A Bayesian test for the hot hand phenomenon. *Journal of Mathematical Psychology*, 72, 200–209. <https://doi.org/10.1016/j.jmp.2015.12.003>
- Yaari, G., & Eisenmann, S. (2011). The hot (invisible?) hand: Can time sequence patterns of success/failure in sports be modeled as repeated random independent trials? *PLOS ONE*, 6(10), e24532. <https://doi.org/10.1371/journal.pone.0024532>

A Reproducibility Code and Additional Implementation Details

The code below gives the main reproducibility steps used for estimation and prediction. Imports, device configuration, tensor padding, batching routines, and file paths are omitted for brevity.

Listing 1: Main reproducibility code for the xFG-adjusted state-space model.

```
# =====
# xFG-adjusted continuous-time state-space model
# =====

import gc
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F

# =====
# General setup
# =====

SEED = 123
np.random.seed(SEED)
torch.manual_seed(SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DTYPE = torch.float32

torch.set_default_dtype(DTYPE)
if DEVICE.type == "cuda":
    torch.backends.cudnn.benchmark = True

# Local data and output directories. The cleaned data files are available on request.
DATA_DIR = Path("path/to/data")
OUTPUT_DIR = Path("path/to/output")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_TAG = "xfg_adjusted"

# =====
# Data loading
# =====

df_2023 = pd.read_parquet(DATA_DIR / "MD_FG_MODEL_B_XFG_GAMECLOCK_2023.parquet")
df_2024 = pd.read_parquet(DATA_DIR / "MD_FG_MODEL_B_XFG_GAMECLOCK_2024.parquet")
df_2025 = pd.read_parquet(DATA_DIR / "MD_FG_MODEL_B_XFG_GAMECLOCK_2025.parquet")

# =====
# Train-test split
# =====

GAME_COL = "game_id"

# Keep complete games together when splitting the 2025 season.
games_2025 = np.array(sorted(df_2025[GAME_COL].unique()))
cut_idx = int(np.floor(0.60 * len(games_2025)))

train_2025_games = set(games_2025[:cut_idx])
test_2025_games = set(games_2025[cut_idx:])

df_2025_train60 = df_2025[df_2025[GAME_COL].isin(train_2025_games)].copy()
df_2025_test40 = df_2025[df_2025[GAME_COL].isin(test_2025_games)].copy()

# =====
# Sample construction
# =====

PLAYER_COL = "player_id"
PLAYER_NAME_COL = "player_name"
SEASON_COL = "season"
SEQUENCE_COL = "sequence_id"
TIME_COL = "t_game_minutes"
OUTCOME_COL = "make"
XFG_COL = "xfg_logit_model"
```

```

COV_COLS = ["scorediff_model", XFG_COL]
GROUP_COLS = [SEASON_COL, SEQUENCE_COL]

MIN_ATTEMPTS_PER_PLAYER = 500
MIN_ATTEMPTS_PER_SEQUENCE = 6
MIN_SEQ = MIN_ATTEMPTS_PER_SEQUENCE

df_train = pd.concat([df_2023, df_2024, df_2025_train60], ignore_index=True)

needed = [
    SEASON_COL,
    SEQUENCE_COL,
    PLAYER_COL,
    PLAYER_NAME_COL,
    GAME_COL,
    TIME_COL,
    OUTCOME_COL,
] + COV_COLS

# Retain players with enough attempts for stable player-level intercepts.
counts = df_train.groupby(PLAYER_COL).size()
keep_players = counts[counts >= MIN_ATTEMPTS_PER_PLAYER].index

df_train = (
    df_train[df_train[PLAYER_COL].isin(keep_players)]
    .reset_index(drop=True)
)

# Keep player-game sequences with enough shots for within-game filtering.
df_train = (
    df_train.groupby(GROUP_COLS, group_keys=False)
    .filter(lambda g: len(g) >= MIN_ATTEMPTS_PER_SEQUENCE)
    .sort_values(GROUP_COLS + [TIME_COL])
    .reset_index(drop=True)
)

# Construct the held-out test set using the same player and sequence filters.
df_test = df_2025_test40[df_2025_test40[PLAYER_COL].isin(keep_players)].copy()

df_test_pred = (
    df_test.groupby(GROUP_COLS, group_keys=False)
    .filter(lambda g: len(g) >= MIN_ATTEMPTS_PER_SEQUENCE)
    .sort_values(GROUP_COLS + [TIME_COL])
    .reset_index(drop=True)
)

# =====
# Sequence tensors for the forward algorithm
# =====

unique_players = sorted(df_train[PLAYER_COL].unique())
pid_to_idx = {pid: i for i, pid in enumerate(unique_players)}
n_players = len(unique_players)

seq_rows = []

for _, grp in df_train.groupby(GROUP_COLS, sort=False):
    t = grp[TIME_COL].to_numpy(dtype=np.float64)
    y = grp[OUTCOME_COL].to_numpy(dtype=np.int64)
    X = grp[COV_COLS].to_numpy(dtype=np.float32)
    p = pid_to_idx[grp[PLAYER_COL].iloc[0]]

    # Zero gaps are allowed when two shots share the same game-clock timestamp.
    delta = np.diff(t) if len(t) > 1 else np.zeros(0, dtype=np.float64)
    delta = np.maximum(delta, 0.0)

    seq_rows.append((y, X, delta, p))

n_seq = len(seq_rows)
max_T = max(len(y) for y, *_ in seq_rows)
n_obs = sum(len(y) for y, *_ in seq_rows)
n_cov = len(COV_COLS)

y_pad = torch.zeros(n_seq, max_T, dtype=torch.int64, device=DEVICE)
X_pad = torch.zeros(n_seq, max_T, n_cov, dtype=DTYPE, device=DEVICE)
delta_pad = torch.zeros(n_seq, max(max_T - 1, 0), dtype=DTYPE, device=DEVICE)
mask_obs = torch.zeros(n_seq, max_T, dtype=torch.bool, device=DEVICE)
p_all = torch.zeros(n_seq, dtype=torch.long, device=DEVICE)

for i, (y, X, delta, p) in enumerate(seq_rows):
    T_i = len(y)

```

```

y_pad[i, :T_i] = torch.as_tensor(y, device=DEVICE)
X_pad[i, :T_i, :] = torch.as_tensor(X, dtype=DTYPE, device=DEVICE)

if T_i > 1:
    delta_pad[i, :T_i - 1] = torch.as_tensor(delta, dtype=DTYPE, device=DEVICE)

mask_obs[i, :T_i] = True
p_all[i] = p

# When the time gap is zero, the state is not propagated; only a new emission is added.
if max_T > 1:
    mask_after_first = mask_obs[:, 1:]
    mask_same = (delta_pad < 1e-8) & mask_after_first
else:
    mask_after_first = torch.zeros(n_seq, 0, dtype=torch.bool, device=DEVICE)
    mask_same = torch.zeros(n_seq, 0, dtype=torch.bool, device=DEVICE)

# =====
# OU state grid
# =====

K = 100
GRID_LO, GRID_HI = -2.0, 2.0

# Approximate the continuous latent state with a finite grid.
grid_edges = torch.linspace(GRID_LO, GRID_HI, K + 1, dtype=DTYPE, device=DEVICE)
grid_c = 0.5 * (grid_edges[:-1] + grid_edges[1:])

GRID_FROM = grid_c.view(1, K, 1)
BIN_LOWER = grid_edges[:-1].view(1, 1, K)
BIN_UPPER = grid_edges[1:].view(1, 1, K)

SQRT2 = torch.sqrt(torch.tensor(2.0, dtype=DTYPE, device=DEVICE))
EPS = 1e-30

def normal_cdf(z):
    """Computes standard normal CDF values using torch operations."""
    return 0.5 * (1.0 + torch.erf(z / SQRT2))

# =====
# Approximate likelihood
# =====

def chunk_nll(start, end, log_theta, log_sigma, beta_0, beta):
    """Computes the approximate negative log-likelihood for a batch of sequences."""
    theta = torch.exp(log_theta)
    sigma = torch.exp(log_sigma)
    stat_sd = sigma / torch.sqrt(2.0 * theta)

    # Stationary initial distribution over state bins.
    p_init = normal_cdf(grid_edges[1:] / stat_sd) - normal_cdf(grid_edges[:-1] / stat_sd)
    log_init = torch.log(p_init.clamp_min(EPS))

    y_b = y_pad[start:end]
    X_b = X_pad[start:end]
    d_b = delta_pad[start:end]
    m_b = mask_obs[start:end]
    same_b = mask_same[start:end]
    p_b = p_all[start:end]

    lp = beta_0[p_b].unsqueeze(1) + X_b @ beta
    logit = grid_c.view(1, 1, K) + lp.unsqueeze(-1)

    log_emit = torch.where(
        y_b.unsqueeze(-1).bool(),
        F.logsigmoid(logit),
        F.logsigmoid(-logit),
    )

    log_alpha = log_init.unsqueeze(0) + log_emit[:, 0, :]

    for tau in range(1, max_T):
        active = m_b[:, tau]

        if not torch.any(active):
            break

        same = same_b[:, tau - 1] & active
        move = (~same_b[:, tau - 1]) & active

```

```

log_alpha_new = log_alpha.clone()

# Same-timestamp observations share the same latent state.
if torch.any(same):
    idx = torch.where(same)[0]
    log_alpha_new[idx] = log_alpha[idx] + log_emit[idx, tau, :]

# Otherwise, propagate the state using the OU transition distribution.
if torch.any(move):
    idx = torch.where(move)[0]
    delta_tau = d_b[idx, tau - 1].view(-1, 1, 1)

    mean = torch.exp(-theta * delta_tau) * GRID_FROM
    var = (sigma ** 2) / (2.0 * theta) * (
        1.0 - torch.exp(-2.0 * theta * delta_tau)
    )
    sd = torch.sqrt(var.clamp_min(1e-12))

    Gamma = normal_cdf((BIN_UPPER - mean) / sd) - normal_cdf(
        (BIN_LOWER - mean) / sd
    )
    log_Gamma = torch.log(Gamma.clamp_min(EPS))

    log_alpha_move = torch.logsumexp(
        log_alpha[idx].unsqueeze(2) + log_Gamma,
        dim=1,
    )

    log_alpha_new[idx] = log_alpha_move + log_emit[idx, tau, :]

log_alpha = log_alpha_new

return -torch.logsumexp(log_alpha, dim=1).sum()

CHUNK_SIZE = 1024

def nll_total(log_theta, log_sigma, beta_0, beta, chunk_size=CHUNK_SIZE):
    """Computes the total negative log-likelihood without storing gradients."""
    total = 0.0

    with torch.no_grad():
        for start in range(0, n_seq, chunk_size):
            end = min(start + chunk_size, n_seq)
            loss = chunk_nll(start, end, log_theta, log_sigma, beta_0, beta)
            total += float(loss.detach().cpu())

    return total

def backward_total(log_theta, log_sigma, beta_0, beta, chunk_size=CHUNK_SIZE):
    """Backpropagates the likelihood in chunks to reduce memory use."""
    total = torch.zeros((), dtype=DTYPE, device=DEVICE)

    for start in range(0, n_seq, chunk_size):
        end = min(start + chunk_size, n_seq)
        loss = chunk_nll(start, end, log_theta, log_sigma, beta_0, beta)
        total = total + loss.detach()
        loss.backward()
        del loss

    return total

# =====
# Parameter estimation
# =====

log_theta = torch.tensor(
    np.log(0.02),
    dtype=DTYPE,
    device=DEVICE,
    requires_grad=True,
)

log_sigma = torch.tensor(
    np.log(0.1),
    dtype=DTYPE,
    device=DEVICE,
    requires_grad=True,
)

```

```

# Initialise player intercepts from empirical field-goal percentages.
pr = df_train.groupby(PLAYER_COL)[OUTCOME_COL].mean().loc[unique_players].values
pr = np.clip(pr, 0.02, 0.98)

beta_0 = torch.tensor(
    np.log(pr / (1.0 - pr)),
    dtype=DTYPE,
    device=DEVICE,
    requires_grad=True,
)

beta = torch.zeros(n_cov, dtype=DTYPE, device=DEVICE, requires_grad=True)

opt = torch.optim.LBFGS(
    [log_theta, log_sigma, beta_0, beta],
    lr=0.1,
    max_iter=50,
    max_eval=80,
    tolerance_grad=1e-3,
    tolerance_change=1e-6,
    history_size=20,
    line_search_fn="strong_wolfe",
)

def closure():
    opt.zero_grad(set_to_none=True)
    loss = backward_total(log_theta, log_sigma, beta_0, beta)
    return loss

gc.collect()

if DEVICE.type == "cuda":
    torch.cuda.empty_cache()

opt.step(closure)

# A short continuation step is used to refine the likelihood optimum.
opt_cont = torch.optim.LBFGS(
    [log_theta, log_sigma, beta_0, beta],
    lr=0.05,
    max_iter=50,
    max_eval=80,
    tolerance_grad=1e-4,
    tolerance_change=1e-7,
    history_size=20,
    line_search_fn="strong_wolfe",
)

def closure_cont():
    opt_cont.zero_grad(set_to_none=True)
    loss = backward_total(log_theta, log_sigma, beta_0, beta)
    return loss

opt_cont.step(closure_cont)

# =====
# No-state benchmark model
# =====

p_flat = torch.from_numpy(
    df_train[PLAYER_COL].map(pid_to_idx).to_numpy(np.int64)
).to(DEVICE)

y_flat = torch.from_numpy(
    df_train[OUTCOME_COL].to_numpy(np.float32)
).to(DEVICE, dtype=DTYPE)

X_flat = torch.from_numpy(
    df_train[COV_COLS].to_numpy(np.float32)
).to(DEVICE, dtype=DTYPE)

pr_b = np.clip(
    df_train.groupby(PLAYER_COL)[OUTCOME_COL].mean().loc[unique_players].values,
    0.02,
    0.98,
)

```

```

bench_beta_0 = torch.tensor(
    np.log(pr_b / (1.0 - pr_b)),
    dtype=DTYPE,
    device=DEVICE,
    requires_grad=True,
)

bench_beta = torch.zeros(n_cov, dtype=DTYPE, device=DEVICE, requires_grad=True)

def benchmark_nll(b0, b):
    """Computes the benchmark logistic-regression negative log-likelihood."""
    logits = b0[p_flat] + X_flat @ b
    return F.binary_cross_entropy_with_logits(logits, y_flat, reduction="sum")

bench_opt = torch.optim.LBFGS(
    [bench_beta_0, bench_beta],
    lr=0.1,
    max_iter=100,
    max_eval=150,
    tolerance_grad=1e-5,
    tolerance_change=1e-8,
    history_size=20,
    line_search_fn="strong_wolfe",
)

def bench_closure():
    bench_opt.zero_grad(set_to_none=True)
    loss = benchmark_nll(bench_beta_0, bench_beta)
    loss.backward()
    return loss

bench_opt.step(bench_closure)

# =====
# In-sample model comparison
# =====

ou_nll = nll_total(log_theta, log_sigma, beta_0, beta)
benchmark_final_nll = benchmark_nll(
    bench_beta_0,
    bench_beta,
).detach().cpu().item()

theta_hat = torch.exp(log_theta).detach().cpu().item()
sigma_hat = torch.exp(log_sigma).detach().cpu().item()
stat_sd_hat = sigma_hat / np.sqrt(2.0 * theta_hat)
half_life_hat = np.log(2.0) / theta_hat

k_benchmark = n_players + n_cov
k_ou = k_benchmark + 2

aic_benchmark = 2 * k_benchmark + 2 * benchmark_final_nll
bic_benchmark = np.log(n_obs) * k_benchmark + 2 * benchmark_final_nll

aic_ou = 2 * k_ou + 2 * ou_nll
bic_ou = np.log(n_obs) * k_ou + 2 * ou_nll

comparison = pd.DataFrame(
    {
        "model": ["benchmark", "OU + xFG"],
        "k": [k_benchmark, k_ou],
        "NLL": [benchmark_final_nll, ou_nll],
        "AIC": [aic_benchmark, aic_ou],
        "BIC": [bic_benchmark, bic_ou],
        "theta": [np.nan, theta_hat],
        "sigma": [np.nan, sigma_hat],
        "stationary_sd": [np.nan, stat_sd_hat],
        "half_life": [np.nan, half_life_hat],
    }
)

comparison.to_csv(
    OUTPUT_DIR / f"in_sample_comparison_{MODEL_TAG}_min{MIN_SEQ}.csv",
    index=False,
)

# =====

```

```

# Held-out next-shot prediction
# =====

@torch.no_grad()
def transition_probs_for_gap(delta_value, theta, sigma):
    """Computes OU transition probabilities for one time gap."""
    delta_value = float(delta_value)

    if delta_value < 1e-8:
        return torch.eye(K, dtype=DTYPE, device=DEVICE)

    stat_sd = sigma / torch.sqrt(2.0 * theta)
    d = torch.tensor(delta_value, dtype=DTYPE, device=DEVICE)

    mean_coef = torch.exp(-theta * d)
    trans_var = stat_sd ** 2 * (1.0 - torch.exp(-2.0 * theta * d))
    trans_sd = torch.sqrt(trans_var.clamp_min(1e-30))

    mean = mean_coef * grid_c.view(K, 1)
    upper = grid_edges[1:].view(1, K)
    lower = grid_edges[:-1].view(1, K)

    p_trans = normal_cdf((upper - mean) / trans_sd) - normal_cdf(
        (lower - mean) / trans_sd
    )

    p_trans = p_trans / p_trans.sum(dim=1, keepdim=True).clamp_min(EPS)
    return p_trans.clamp_min(EPS)

@torch.no_grad()
def predict_one_step_sequence(grp):
    """Predicts each shot using only earlier shots from the same sequence."""
    grp = grp.sort_values(TIME_COL, kind="mergesort").copy()

    y_np = grp[OUTCOME_COL].to_numpy(np.int64)
    X_np = grp[COV_COLS].to_numpy(np.float32)
    t_np = grp[TIME_COL].to_numpy(np.float64)

    player_idx = pid_to_idx[grp[PLAYER_COL].iloc[0]]
    T = len(grp)

    theta = torch.exp(log_theta.detach())
    sigma = torch.exp(log_sigma.detach())
    stat_sd = sigma / torch.sqrt(2.0 * theta)

    # Each held-out player-game starts from the stationary prior.
    alpha = normal_cdf(grid_edges[1:] / stat_sd) - normal_cdf(grid_edges[:-1] / stat_sd)
    alpha = alpha / alpha.sum().clamp_min(EPS)

    preds_ou = []
    preds_bench = []
    filt_mean = []
    filt_sd = []

    X_t = torch.from_numpy(X_np).to(DEVICE, dtype=DTYPE)

    for tau in range(T):
        x_tau = X_t[tau]

        lp_ou = beta_0[player_idx] + x_tau @ beta
        lp_bench = bench_beta_0[player_idx] + x_tau @ bench_beta

        # Store the filtered state before observing the current shot.
        m = torch.sum(alpha * grid_c)
        v = torch.sum(alpha * (grid_c - m) ** 2)

        filt_mean.append(m.item())
        filt_sd.append(torch.sqrt(v.clamp_min(0)).item())

        p_make_by_state = torch.sigmoid(lp_ou + grid_c)

        preds_ou.append(torch.sum(alpha * p_make_by_state).item())
        preds_bench.append(torch.sigmoid(lp_bench).item())

        # Update on the observed make or miss.
        emission = p_make_by_state if y_np[tau] == 1 else (1.0 - p_make_by_state)
        alpha = alpha * emission
        alpha = alpha / alpha.sum().clamp_min(EPS)

        # Propagate the filtered distribution to the next shot time.
        if tau < T - 1:
            d = t_np[tau + 1] - t_np[tau]

```

```

        P = transition_probs_for_gap(d, theta, sigma)
        alpha = alpha @ P
        alpha = alpha / alpha.sum().clamp_min(EPS)

    out = grp.copy()
    out["shot_number_in_sequence"] = np.arange(1, T + 1)
    out["pred_prob_ou"] = preds_ou
    out["pred_prob_benchmark"] = preds_bench
    out["filtered_state_mean_before_shot"] = filt_mean
    out["filtered_state_sd_before_shot"] = filt_sd

    return out

pred_dfs = []

for _, grp in df_test_pred.sort_values(GROUP_COLS + [TIME_COL]).groupby(
    GROUP_COLS,
    sort=False,
):
    pred_dfs.append(predict_one_step_sequence(grp))

predictions_df = pd.concat(pred_dfs, ignore_index=True)

def evaluate_predictions(df, sample_name):
    """Computes held-out NLL and Brier scores for OU and benchmark predictions."""
    y = df[OUTCOME_COL].to_numpy(float)

    p_ou = np.clip(df["pred_prob_ou"].to_numpy(float), 1e-12, 1 - 1e-12)
    p_bench = np.clip(
        df["pred_prob_benchmark"].to_numpy(float),
        1e-12,
        1 - 1e-12,
    )

    ou_nll_t = -np.sum(y * np.log(p_ou) + (1.0 - y) * np.log(1.0 - p_ou))
    bench_nll_t = -np.sum(
        y * np.log(p_bench) + (1.0 - y) * np.log(1.0 - p_bench)
    )

    benchmark_brier = np.mean((p_bench - y) ** 2)
    ou_brier = np.mean((p_ou - y) ** 2)

    return {
        "sample": sample_name,
        "model_tag": MODEL_TAG,
        "min_shots_per_sequence": MIN_SEQ,
        "n_shots": len(df),
        "n_sequences": df.groupby(GROUP_COLS).ngroups,
        "n_players": df[PLAYER_COL].nunique(),
        "benchmark_test_nll": bench_nll_t,
        "ou_test_nll": ou_nll_t,
        "test_nll_improvement": bench_nll_t - ou_nll_t,
        "test_nll_improvement_per_shot": (bench_nll_t - ou_nll_t) / len(df),
        "benchmark_test_nll_per_shot": bench_nll_t / len(df),
        "ou_test_nll_per_shot": ou_nll_t / len(df),
        "benchmark_brier": benchmark_brier,
        "ou_brier": ou_brier,
        "brier_improvement": benchmark_brier - ou_brier,
    }

m_all = evaluate_predictions(predictions_df, "all_test_shots")

m_after = evaluate_predictions(
    predictions_df[predictions_df["shot_number_in_sequence"] >= 2].copy(),
    "after_first_shot_only",
)

prediction_metrics_df = pd.DataFrame([m_all, m_after])

predictions_df.to_csv(
    OUTPUT_DIR / f"next_shot_predictions_{MODEL_TAG}_min{MIN_SEQ}.csv",
    index=False,
)

prediction_metrics_df.to_csv(
    OUTPUT_DIR / f"next_shot_metrics_{MODEL_TAG}_min{MIN_SEQ}.csv",
    index=False,
)
\begin{lstlisting}[style=appendixcode, language=Python, caption={Expected field-goal (xFG) model: fitting, evaluation, and 2023 hold-out calibration.}, label={lst:xfg_code}]

```

```

# =====
# Expected field-goal (xFG) logistic model
# =====
import ast
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
from patsy import dmatrices
from sklearn.metrics import (
    roc_auc_score, brier_score_loss, log_loss,
    accuracy_score, confusion_matrix,
)

# Local data files. The cleaned data files are available on request.
DATA_PATH_2022 = "cleaned_df_2022.parquet"
DATA_PATH_2023 = "cleaned_df_2023.parquet"

# =====
# Data loading and feature engineering
# =====
df = pd.read_parquet(DATA_PATH_2022)
fg = df[df['Field Goal'] == 1].copy()

# Some rows store court co-ordinates as the string "[x, y]" rather than a list.
def parse_pair(c, to_type=float):
    if isinstance(c, (list, np.ndarray)):
        return to_type(c[0]), to_type(c[1])
    try:
        p = ast.literal_eval(str(c))
        return to_type(p[0]), to_type(p[1])
    except Exception:
        return np.nan, np.nan

coords = fg['Co-ordinate (x, y)'].apply(parse_pair)
fg['x_raw'] = [c[0] for c in coords]
fg['y_raw'] = [c[1] for c in coords]

# Angular offset from the centre of the court (y = 50 is dead centre).
fg['y_offcenter'] = (fg['y_raw'] - 50).abs()

# Shot type: 5 categories, reference = 2pt_jumper.
def shot_type(ev_str):
    if '3pt' in ev_str: return '3pt_jumper'
    if 'Layup' in ev_str: return '2pt_layup'
    if 'DUNK' in ev_str: return '2pt_dunk'
    if 'Hook' in ev_str: return '2pt_hook'
    if 'Jump' in ev_str: return '2pt_jumper'
    return '2pt_jumper'

fg['shot_type'] = fg['Event'].astype(str).apply(shot_type)

# Action group: 12 categories, reference = 'none' (catch-and-shoot / untagged).
def action_group(a):
    if pd.isna(a): return 'none'
    a = str(a).lower().strip()
    if 'alley-oop' in a: return 'alley_oop'
    if 'putback' in a or a == 'tip': return 'putback_tip'
    if 'cutting' in a: return 'cutting'
    if 'running' in a: return 'transition'
    if 'reverse' in a: return 'reverse'
    if 'step back' in a: return 'stepback'
    if 'fadeaway' in a or 'turnaround' in a: return 'fade_turn'
    if 'floating' in a or 'floater' in a: return 'floater'
    if 'pullup' in a: return 'pullup'
    if 'finger roll' in a: return 'finger_roll'
    if 'driving' in a: return 'driving'
    if 'bank' in a: return 'bank'
    return 'none'

fg['action_grp'] = fg['Action Description'].apply(action_group)

# Other covariates.
fg['final_30s'] = fg['Final 30s'].fillna(0).astype(int)
fg['home'] = (fg['Home Team'] == 1).astype(int)
fg['distance'] = fg['Shot Distance'].astype(float)

fg = fg.dropna(subset=['Shot Result']).copy()
fg['y'] = fg['Shot Result'].astype(int)

```

```

# =====
# Model fitting
# =====
formula = (
    "y ~ distance"
    " + C(shot_type, Treatment(reference='2pt_jumper'))"
    " + C(action_grp, Treatment(reference='none'))"
    " + y_offcenter"
    " + final_30s"
    " + home"
)

y, X = dmatrices(formula, data=fg, return_type='dataframe')
model = sm.Logit(y, X)
result = model.fit(dispen=0, maxiter=200)

fg['xFG'] = result.predict(X)

coef_table = pd.DataFrame({
    'estimate': result.params,
    'std_err': result.bse,
    'z': result.tvalues,
    'p_value': result.pvalues,
    'ci_low': result.conf_int()[0],
    'ci_high': result.conf_int()[1],
    'odds_ratio': np.exp(result.params),
}).round(4)

# =====
# In-sample evaluation
# =====
y_true = fg['y'].values
p_hat = fg['xFG'].values
y_pred = (p_hat >= 0.5).astype(int)

acc = accuracy_score(y_true, y_pred)
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

print(f"n = {len(y_true):,}")
print(f"Accuracy (0.5 cutoff): {acc:.5f}")
print(f"AUC: {roc_auc_score(y_true, p_hat):.4f}")
print(f"Brier: {brier_score_loss(y_true, p_hat):.4f}")
print(f"Log-loss: {log_loss(y_true, p_hat):.4f}")
print(f"Sensitivity: {tp/(tp+fn):.4f} | Specificity: {tn/(tn+fp):.4f}")

# In-sample calibration by predicted-probability decile.
fg['decile'] = pd.qcut(fg['xFG'], 10, labels=False, duplicates='drop')
calib = fg.groupby('decile').agg(
    n=('y', 'size'),
    predicted=('xFG', 'mean'),
    actual=('y', 'mean'),
).round(3)
calib['gap'] = (calib['actual'] - calib['predicted']).round(3)

# =====
# Out-of-sample evaluation on the 2023 season
# =====
df23 = pd.read_parquet(DATA_PATH_2023)
fg23 = df23[df23['Field Goal'] == 1].copy()

# Apply the same feature pipeline used in training.
coords = fg23['Co-ordinate (x, y)'].apply(parse_pair)
fg23['x_raw'] = [c[0] for c in coords]
fg23['y_raw'] = [c[1] for c in coords]
fg23['y_offcenter'] = (fg23['y_raw'] - 50).abs()

fg23['shot_type'] = fg23['Event'].astype(str).apply(shot_type)
fg23['action_grp'] = fg23['Action Description'].apply(action_group)
fg23['final_30s'] = fg23['Final 30s'].fillna(0).astype(int)
fg23['home'] = (fg23['Home Team'] == 1).astype(int)
fg23['distance'] = fg23['Shot Distance'].astype(float)

fg23 = fg23.dropna(subset=['Shot Result']).copy()
fg23['y'] = fg23['Shot Result'].astype(int)

# Score 2023 shots using the 2022-fitted model.
y23, X23 = dmatrices(formula, data=fg23, return_type='dataframe')
X23 = X23.reindex(columns=result.params.index, fill_value=0)
fg23['xFG'] = result.predict(X23)

y_true_23 = fg23['y'].values
p_23 = fg23['xFG'].values
y_pred_23 = (p_23 >= 0.5).astype(int)

```

```

print(f"Out-of-sample (2023): n = {len(y_true_23):,}")
print(f"Accuracy: {accuracy_score(y_true_23, y_pred_23):.5f}")
print(f"AUC: {roc_auc_score(y_true_23, p_23):.4f}")
print(f"Brier: {brier_score_loss(y_true_23, p_23):.4f}")
print(f"Log-loss: {log_loss(y_true_23, p_23):.4f}")

# =====
# Calibration plot for the held-out 2023 season
# =====
fg23['xFG_decile'] = pd.qcut(fg23['xFG'], 10, labels=False, duplicates='drop')

xfg_calib = (
    fg23
    .groupby('xFG_decile')
    .agg(shots=('y', 'size'),
         mean_xfg=('xFG', 'mean'),
         observed_make_rate=('y', 'mean'))
    .reset_index()
)
xfg_calib['gap'] = xfg_calib['observed_make_rate'] - xfg_calib['mean_xfg']

plt.figure(figsize=(6, 4))
plt.plot(xfg_calib['mean_xfg'], xfg_calib['observed_make_rate'],
         marker='o', label='Observed make rate')

lo = min(xfg_calib['mean_xfg'].min(), xfg_calib['observed_make_rate'].min())
hi = max(xfg_calib['mean_xfg'].max(), xfg_calib['observed_make_rate'].max())
plt.plot([lo, hi], [lo, hi], ls='--', lw=1, label='Perfect calibration')

plt.xlabel('Mean predicted xFG')
plt.ylabel('Observed make rate')
plt.title('xFG calibration on 2023 hold-out')
plt.legend()
plt.tight_layout()
plt.savefig('xfg_calibration_2023.pdf', bbox_inches='tight')

```